Kubernetes in Action

Chapter 1: 쿠버네티스 소개

Kubernetes in Action 1st Edition — Marko Lukša

목차

1. **쿠버네티스가 필요한 이유** — Monolith → Microservices
2. **컨테이너 기술** — VM vs Container, Docker
3. **쿠버네티스 아키텍처** — Control Plane & Worker Node 상세
4. **Self-managed vs Managed K8s** — EKS / GKE / AKS
5. **애플리케이션 실행 흐름** — 선언적 모델
6. **서비스 디스커버리 Deep Dive** — DNS → Eureka → kube-proxy + CoreDNS
7. **Traditional MSA vs Kubernetes** — 무엇이 바뀌었는가
8. **요약**

1. 왜 쿠버네티스가 필요한가?

Monolith의 한계

```
┌──────────────────────────┐
│      Monolith App        │
│  ┌────┬────┬────┬────┐   │
│  │ UI │Ord.│Pay │Inv │   │
│  └────┴────┴────┴────┘   │
│   하나의 프로세스 실행   │
└──────────────────────────┘
```

- 모든 컴포넌트가 **하나의 단위**로 개발, 배포, 스케일링
- **수직 스케일링**: 더 큰 서버 (비용↑, 물리적 한계)
- **수평 스케일링**: 전체를 복제 (리소스 낭비)
- 하나의 버그가 **전체 시스템**을 다운시킬 수 있음

Microservices로의 전환

장점: 개별 개발, 배포, 스케일링 가능

새로운 문제:

- 🔍 서비스끼리 **서로를 어떻게 찾는가?** (Service Discovery)
- 🔗 **내부 통신**은 어떻게 하는가?
- ⚙️ 서로 다른 **라이브러리 버전** 충돌

DevOps → NoOps

- DevOps: 개발 팀이 배포 + 운영까지 책임
- NoOps: 인프라가 완전히 추상화 → 개발자는 앱만 제출

> 쿠버네티스가 이 NoOps를 가능하게 한다.
> 개발자는 앱을 제출하고, K8s가 어디서 실행할지 결정한다.

2. 컨테이너 기술

VM vs Container

VM 방식

```
App A
Guest OS
App B
Guest OS
Hypervisor
Host OS
Hardware
```

Container 방식

```
App A | B | C
Container
  Runtime
Host OS
Hardware
```

VM vs Container — 비교표

	가상머신	컨테이너
격리	완벽 (별도 커널)	프로세스 수준 (커널 공유)
오버헤드	GB 단위 RAM	MB 단위
시작 시간	분 단위	**초 단위**
밀도	호스트당 수~십 개	호스트당 **수백 개**
보안	강함	커널 공유 리스크
이식성	이미지 크기 큼	레이어 공유로 **효율적**

컨테이너 격리 메커니즘

Linux Namespaces — "무엇을 볼 수 있는가"

각 컨테이너에 **독립된 시스템 뷰** 제공

Namespace	격리 대상
Mount	파일시스템
PID	프로세스 ID
Network	네트워크 인터페이스
IPC	프로세스 간 통신
UTS	호스트명
User	사용자 ID

Linux cgroups — "얼마나 쓸 수 있는가"

CPU, 메모리, 디스크 I/O, 네트워크 **사용량 제한**

Docker

Build → Ship → Run

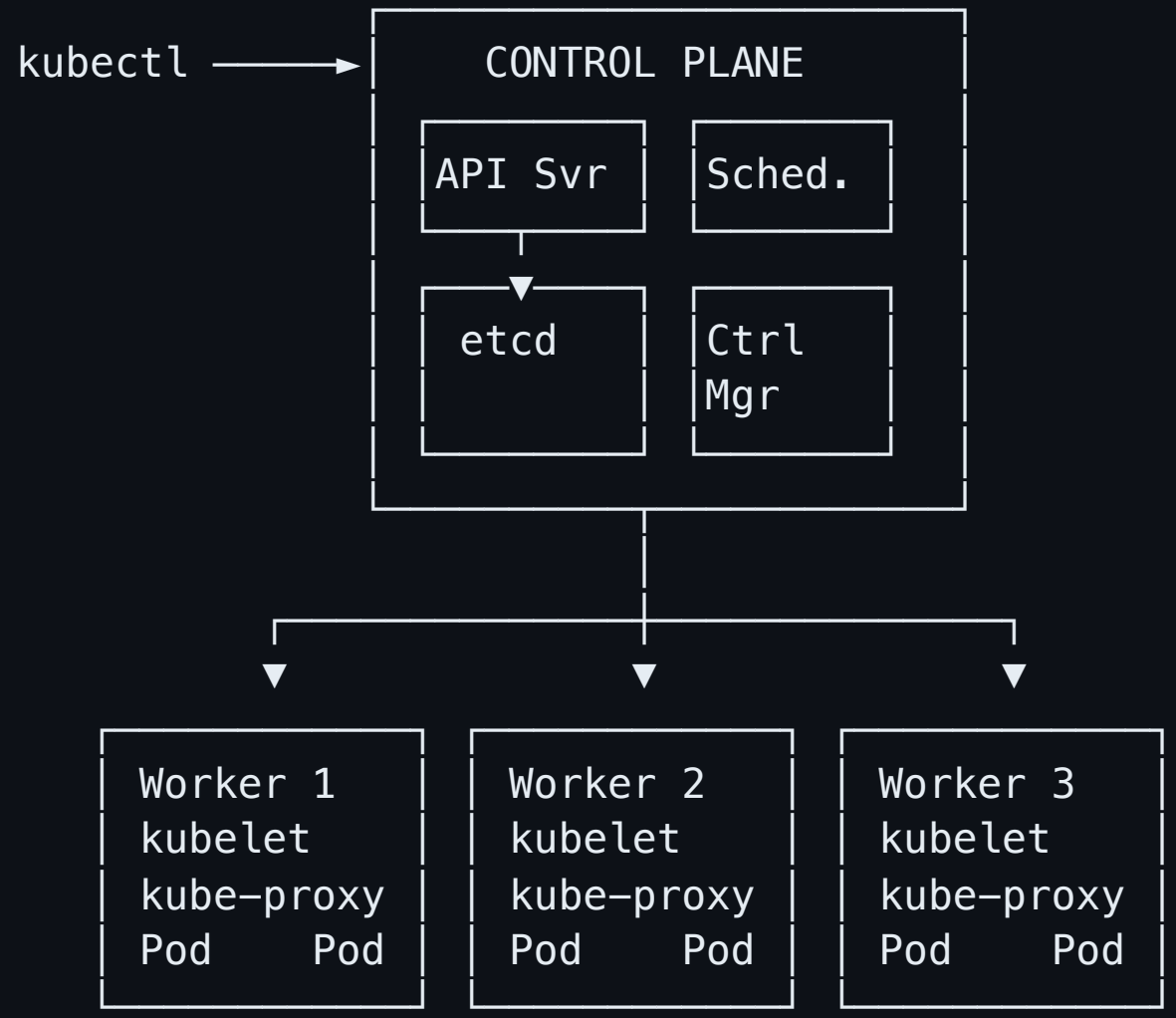
```
Developer          Registry          Production

┌──────────┐      ┌──────────┐      ┌──────────┐
│ docker   │──────►│ Docker   │──────►│ docker   │
│ build    │ push │  Hub     │ pull │ run      │
└──────────┘      └──────────┘      └──────────┘
```

- **이미지**: 앱 + 환경을 레이어로 패키징
- **레지스트리**: 이미지 저장/배포 (Docker Hub)
- **컨테이너**: 이미지의 실행 인스턴스
- **레이어 공유**: 여러 이미지가 동일 레이어 재사용 → 저장/전송 효율

> Docker는 격리를 제공하는 것이 아니라,
> 커널의 **Namespace + cgroups**를 쉽게 쓸 수 있게 해주는 플랫폼이다.

3. 쿠버네티스 아키텍처

전체 구조


kubectl
CONTROL PLANE
API Svr
Sched.
etcd
Ctrl Mgr
Worker 1
kubelet
kube-proxy
Pod Pod
Worker 2
kubelet
kube-proxy
Pod Pod
Worker 3
kubelet
kube-proxy
Pod Pod

Control Plane — 구성요소 상세

구성요소	역할	죽으면?
API Server	유일한 진입점, 인증→인가→Admission→etcd	kubectl 불가, 새 스케줄링 불가
etcd	모든 상태 저장 (Raft, Strong Consistency)	**클러스터 전체 상태 소실**
Scheduler	Filtering→Scoring으로 Pod 배치 결정	새 Pod 배치 불가
Controller Manager	Reconciliation Loop 묶음	Pod 재생성 안 됨
Cloud Controller	클라우드 API 연동 (LB 프로비저닝)	LoadBalancer 생성 불가

> 모든 구성요소가 죽어도 **이미 실행 중인 Pod는 계속 동작**한다.
> kubelet이 독립적으로 컨테이너를 유지하기 때문.

kube-apiserver — 유일한 진입점

```
kubectl ──►
kubelet ──► kube-apiserver ◄──► etcd
scheduler ►        │
controller ►       │
             인증 → 인가 → Admission → etcd 저장
```

- **RESTful API**: 모든 K8s 리소스를 CRUD하는 HTTP API
- **인증(AuthN)**: 요청자가 누구인지 (인증서, 토큰, OIDC)
- **인가(AuthZ)**: 해당 작업 권한이 있는지 (RBAC)
- **Admission Control**: 요청을 변형/거부하는 플러그인 체인
- **Watch**: 리소스 변경을 실시간 구독 (Controller, kubelet 모두 사용)

etcd — 클러스터의 뇌

```
etcd에 저장되는 것:
├── /registry/pods/default/nginx-abc
├── /registry/services/default/my-svc
├── /registry/deployments/...
├── /registry/secrets/...
└── /registry/nodes/worker-1
```

- **Raft 합의**: 3대 → 1대 장애 허용, 5대 → 2대 장애 허용
- **Strong Consistency**: 쓰기 확정 → 어느 노드에서 읽어도 동일
- **API Server만 접근**: 다른 모든 구성요소는 API Server를 거침
- **백업이 가장 중요**: etcd 소실 = 클러스터 전체 상태 소실

kube-scheduler — Pod를 어디에?

```
새 Pod 생성 요청
      │
 1. Filtering (걸러내기)
      │  리소스 부족 노드 제외
      │  Taint/Toleration 불일치 제외
      │  NodeSelector 불일치 제외
      ▼
 2. Scoring (점수 매기기)
      │  리소스 균형 (여유 노드 선호)
      │  Pod Affinity/Anti-Affinity
      │  각 노드에 0~100점
      ▼
 3. Binding (결정)
      │  최고 점수 노드에 할당
      └──► API Server에 기록
```

> Scheduler는 **배치만 결정**. 실제 컨테이너 실행은 kubelet이 한다.

kube-controller-manager — Reconciliation Loop

하나의 프로세스 안에 **여러 독립 컨트롤러**가 동작:

컨트롤러	하는 일
ReplicaSet	replicas: 3인데 2개 → 1개 추가 생성
Deployment	Rolling Update 관리 (새 RS 생성 → 이전 축소)
Node	노드 응답 없으면 → NotReady → Pod 재스케줄링
EndpointSlice	Service의 Pod 목록 관리 (서비스 디스커버리 핵심)

```
while true:
    desired = API Server에서 원하는 상태
    actual  = 현재 상태
    if desired != actual:
        수정 요청 (create/delete/update)
```

Worker Node 구성요소

kubelet — 노드의 에이전트

- 자기 노드에 할당된 Pod를 감지 → Container Runtime에 실행 지시
- Liveness/Readiness/Startup Probe 실행
- 노드 상태를 주기적으로 API Server에 보고
- Static Pod: 로컬 파일로 Pod 실행 (CP 구성요소가 이 방식)

> kubelet은 **유일하게 컨테이너가 아닌 시스템 프로세스**로 실행.
> 컨테이너를 관리하는 주체는 자기 자신이 컨테이너일 수 없다.

kube-proxy — 서비스 네트워킹

- Service/EndpointSlice 변경 감지 → iptables/IPVS 규칙 업데이트
- iptables (기본): O(n), 소규모 / IPVS: O(1), 대규모

4. Self-managed vs Managed K8s

컨트롤 플레인을 누가 관리하는가?

	Self-managed	Managed (EKS/GKE/AKS)
API Server	직접 설치, HA 구성	클라우드가 운영
etcd	직접 클러스터링, 백업	**완전 관리** (접근 불가)
Scheduler / CM	직접 설치	클라우드가 운영
CP HA	직접 multi-master	**기본 HA** (multi-AZ)
K8s 업그레이드	직접 (다운타임 리스크)	콘솔/CLI로 자동 롤링
Worker Nodes	직접 프로비저닝	사용자 관리 (또는 Fargate)

EKS/GKE에서 `kubectl get nodes` → **Worker만** 보인다.
Control Plane은 클라우드 인프라에 숨겨져 접근 불가.

추상화 수준 스펙트럼

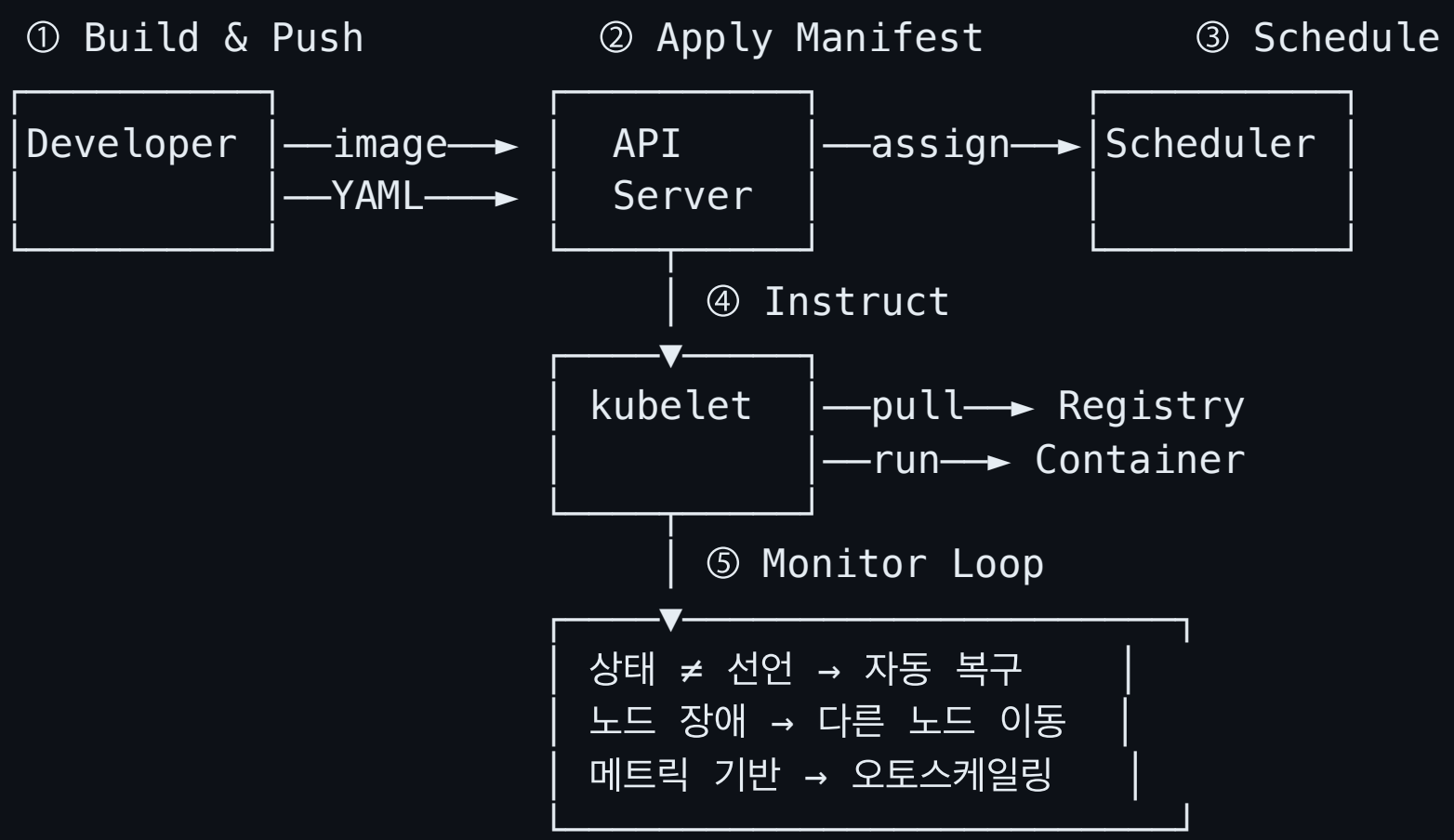
```
관리 부담 높음 ◄──────────────────────────► 관리 부담 낮음

 kubeadm        EKS +        ECS +        GKE
 (직접구축)     EC2 Nodes    Fargate      Autopilot
    |              |            |            |
 CP + Worker     Worker만      Pod만       Pod만 +
 모두 관리       관리          관리        노드도 자동
```

- **ECS Fargate**: 서버리스, 노드 관리 완전 불필요
- **GKE Autopilot**: 노드 자동 관리 + Pod 단위 과금

> 내부 구조를 아는 것이 중요한 이유:
> Managed를 쓰더라도 **디버깅, 성능 튜닝, 아키텍처 결정** 시 필수.

5. 애플리케이션 실행 흐름

배포 프로세스


① Build & Push
② Apply Manifest
③ Schedule
Developer
image
YAML
API Server
assign
Scheduler
④ Instruct
kubelet
pull
Registry
run
Container
⑤ Monitor Loop
상태 ≠ 선언 → 자동 복구
노드 장애 → 다른 노드 이동
메트릭 기반 → 오토스케일링

선언적 모델 (Declarative)

> "**이 상태를 유지해라**"라고 선언하면, K8s가 알아서 맞춘다.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
spec:
  replicas: 3                # ← 항상 3개 유지
  template:
    spec:
      containers:
      - name: order
        image: order-service:1.0
```

- Pod 죽으면 → **자동 재생성** / 노드 다운 → **다른 노드에서 재시작**
- CPU 임계치 초과 → **오토스케일링**

6. 서비스 디스커버리 Deep Dive

왜 서비스 디스커버리가 필요한가?

MSA에서 상대 서비스의 IP:Port를 알아야 한다. 클라우드에서 IP는 동적이다.

```
정적 설정 → DNS → Client-Side → Server-Side → K8s → Service Mesh
    |          |          |            |          |          |
 스케일      캐시로     JVM         LB가       L4만      L7 전부
 불가       stale     종속        SPOF       지원      근데 복잡
```

DNS가 실패하는 이유 — 4중 캐시

캐시 레이어	기본 캐시 시간
JVM InetAddress	∞ (영원히) — SecurityManager 시
OS (nscd)	3,600s ~ 86,400s
DNS Resolver	TTL값 (30~3600s)
브라우저	60s (하드코딩)

그리고 **커넥션 풀이 DNS를 완전히 무시**한다:

- TCP 연결이 맺어지면 DNS 재조회 안 함 (Apache HttpClient, OkHttp)
- 실질적 stale window = `max(JVM TTL, 커넥션 풀 TTL)` = **분 ~ 무한대**

> DNS TTL을 5초로 줘도, JVM이 영원히 캐싱하면 **의미 없다**.

Eureka의 치명적 문제들

Self-Preservation — 좀비 인스턴스

```
인스턴스 10개 중 2개 죽음 → heartbeat 80% < 85%
→ Self-preservation 발동
→ 죽은 인스턴스가 레지스트리에 영구 유지 (좀비)
→ 트래픽이 좀비에게 계속 전달
```

죽은 인스턴스 제거 worst-case: 5분 30초

```
T+0s     인스턴스 종료
T+180s   Lease 만료 (90s × 2, renew 버그)
T+240s   EvictionTask (60s 간격)
T+270s   서버 캐시 갱신 (30s)
T+300s   클라이언트 캐시 (30s)
T+330s   Ribbon 캐시 (30s)
```

K8s가 해결한 방법 — kube-proxy + CoreDNS

ClusterIP — DNS 캐시 문제를 구조적으로 해소

```
CoreDNS: svc-b → 10.96.100.1 (ClusterIP, 고정!)
                      │
                kube-proxy (커널)
                 iptables/IPVS
                ┌──────┼──────┐
                ▼      ▼      ▼
              Pod A  Pod B  Pod C
            (IP는 수시로 변경)
```

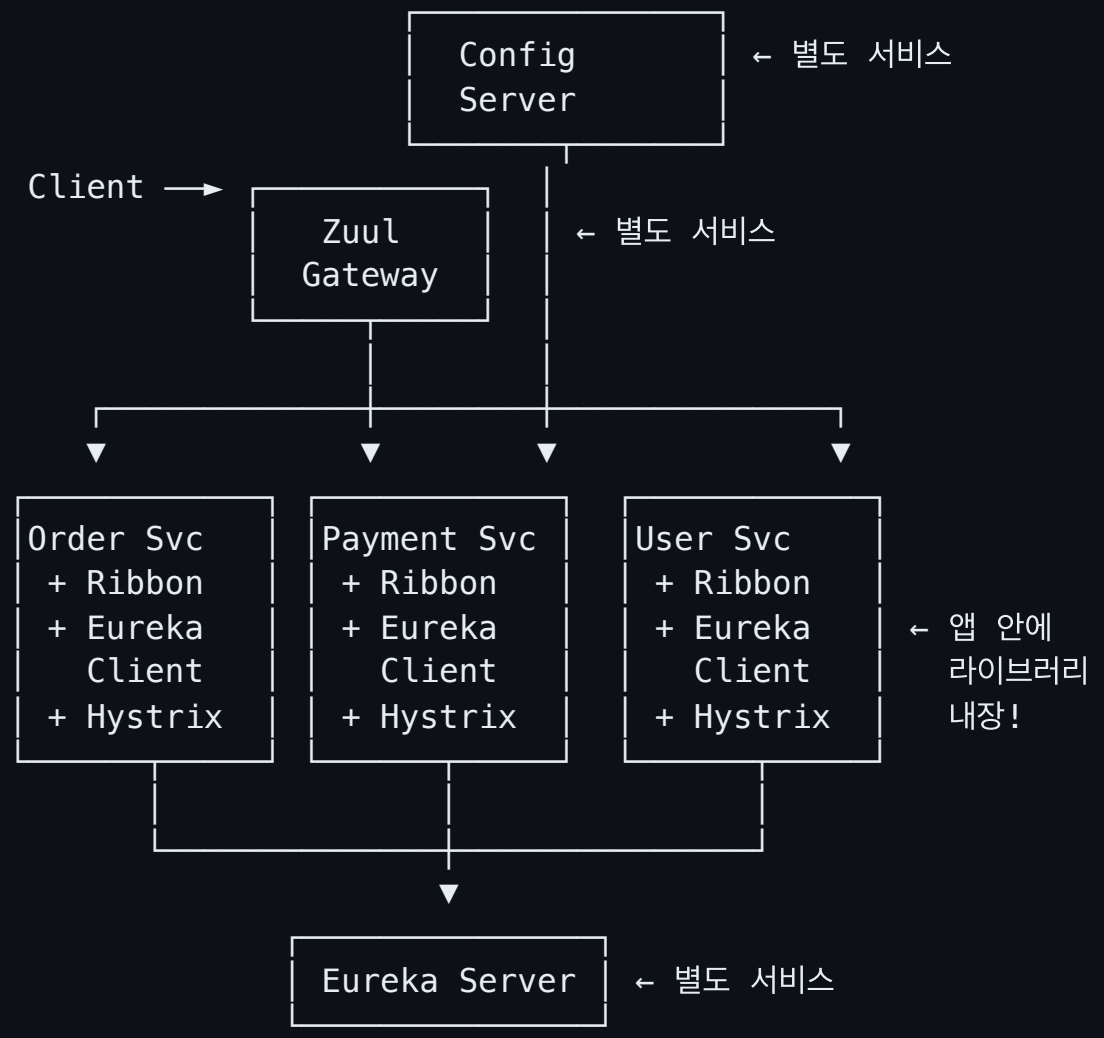
- **CoreDNS**: 서비스 이름 → ClusterIP(고정 VIP) 변환하는 클러스터 DNS
- **kube-proxy**: 각 노드에서 ClusterIP → 실제 Pod IP로 커널 레벨 라우팅
- ClusterIP는 **절대 안 바뀜** → DNS 캐시가 stale해도 무관
- **커넥션 풀 문제 해소**: 커널이 매 패킷마다 실제 Pod로 라우팅

Eureka vs kube-proxy + CoreDNS — 비교

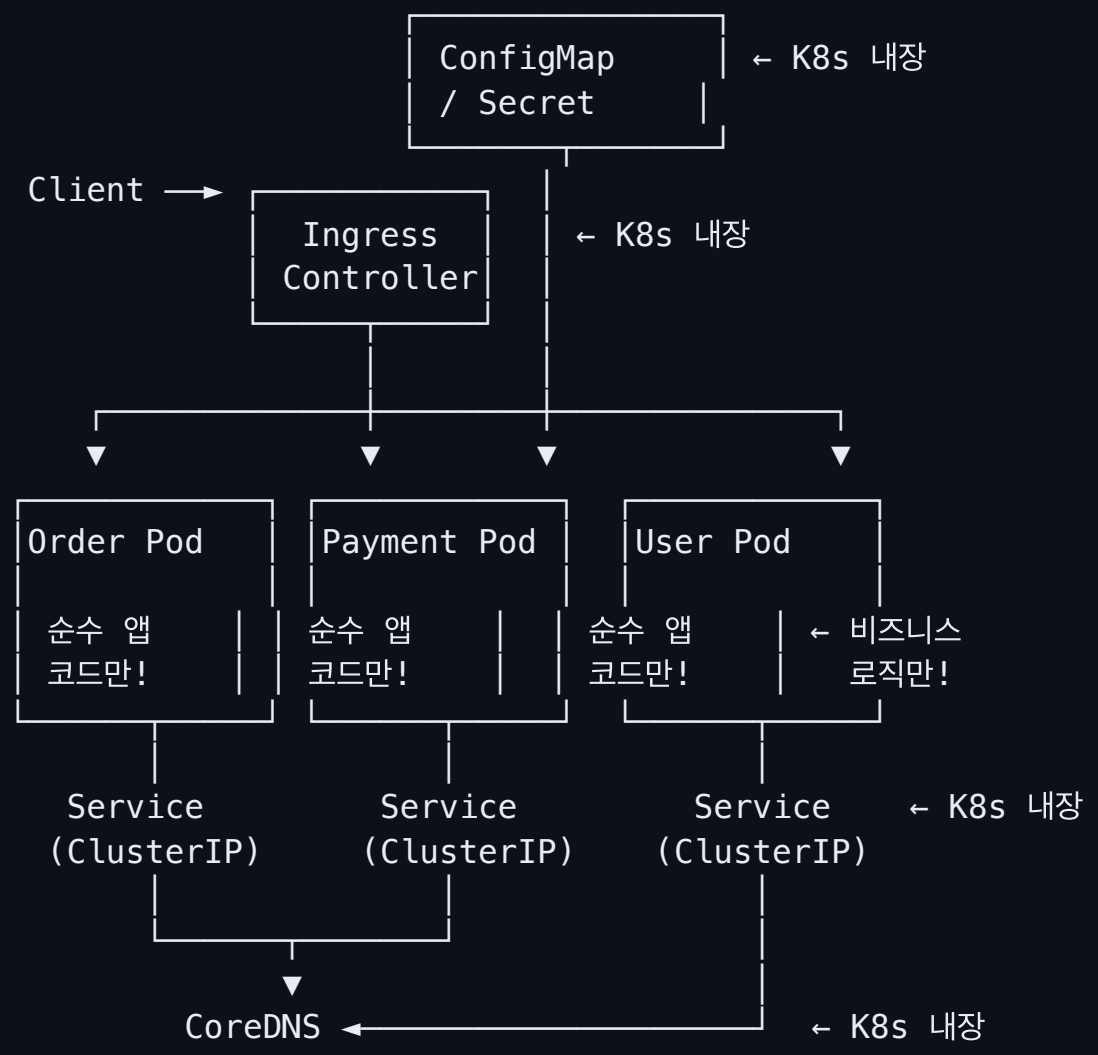
시나리오	Eureka	kube-proxy + CoreDNS
Graceful 종료	30~330초	**~2-5초**
Ungraceful 종료	180~330초	**~30초**
좀비 인스턴스	Self-preservation → 무한 유지	**불가능** (Probe 지속 검증)
레지스트리 SPOF	Eureka 클러스터 장애 가능	**없음** (kube-proxy가 전 노드 분산)
DNS 캐시 문제	5중 캐시 체인	**무관** (ClusterIP 고정)

7. Traditional MSA vs Kubernetes

기존 MSA 아키텍처 (Spring Cloud Netflix)



Kubernetes MSA 아키텍처

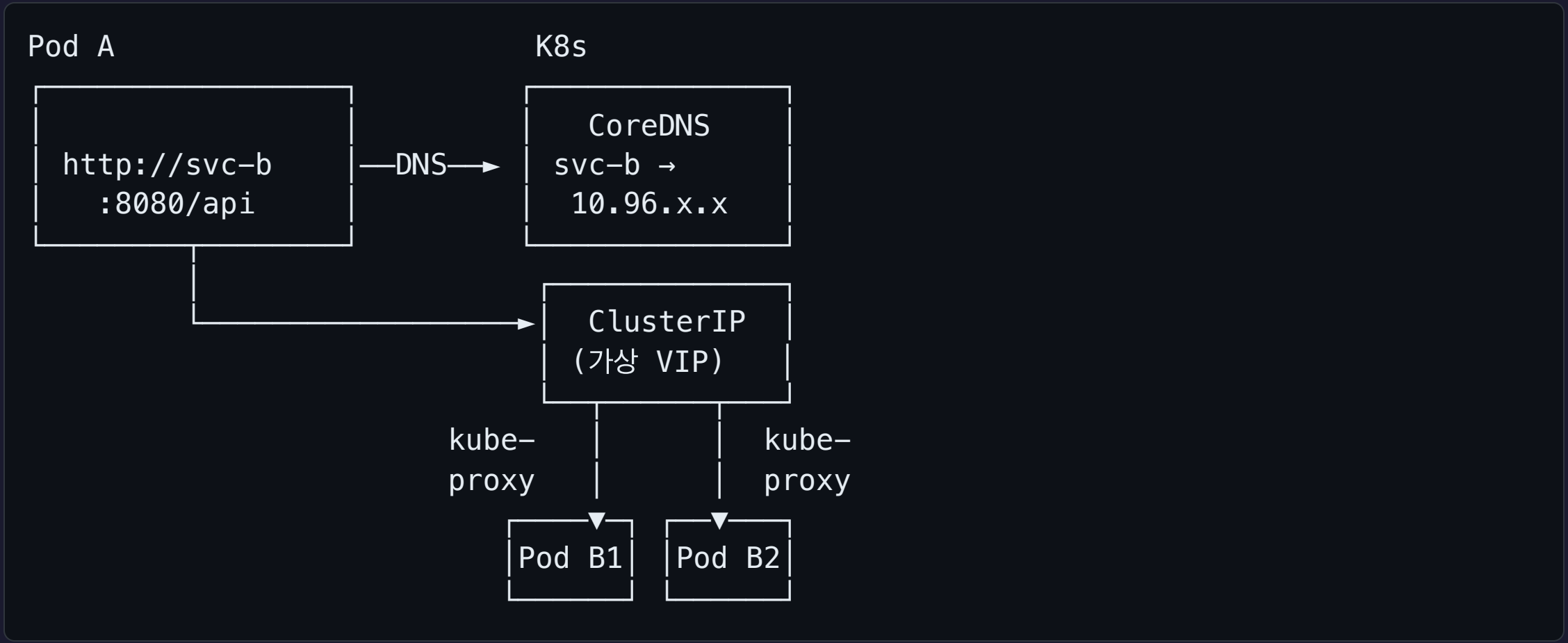


서비스 디스커버리 — Before: Eureka

```
Service A                          Eureka Server

┌──────────────────┐
│ EurekaClient     │──reg──►  ┌──────────────────┐
│ Ribbon           │◄─list──  │ 서비스 목록      │
│ DiscoveryClient  │          │                  │
│    .getInstances │          │  30s 주기        │
└──────────────────┘          │  heartbeat       │
          │                   └──────────────────┘
          │ client-side LB
          ▼
     Service B (IP 직접 호출)
```

- 모든 서비스에 **Eureka Client + Ribbon** 내장 (JVM 종속)
- Eureka Server **별도 운영** 필요
- heartbeat 누락 시 **최대 90초** stale 엔트리 유지

서비스 디스커버리 — After: kube-proxy + CoreDNS



- **CoreDNS**: 서비스 이름 → 고정 ClusterIP 반환 (언어 무관)
- **kube-proxy**: ClusterIP → 실제 Pod IP 커널 레벨 라우팅 (iptables/IPVS)

서비스 디스커버리 — 비교

	Eureka + Ribbon	kube-proxy + CoreDNS
이름 해석	Eureka Server (별도 운영)	**CoreDNS** (클러스터 내장)
로드 밸런싱	Ribbon (앱 내 JVM 라이브러리)	**kube-proxy** (커널 iptables/IPVS)
언어 지원	JVM 중심	**언어 무관 (DNS)**
Stale 엔트리	최대 **90초**	**즉시 갱신**
앱 코드 결합	`DiscoveryClient` 필요	**제로 — DNS만 사용**
장애 시	Registry 다운 = 디스커버리 불가	**kube-proxy가 전 노드 분산**

코드 변화:

```
discoveryClient.getInstances("order") → http://order-service:8080
```

내부 통신 & 로드밸런싱

	Ribbon (in-process)	kube-proxy (kernel)
LB 위치	앱 프로세스 **내부**	커널 레벨 (iptables/IPVS)
캐시	Eureka 전파 지연 → stale	API Server **직접 watch**
성능	앱 스레드 사용	O(1) IPVS 조회
L7 라우팅	Zuul (thread-per-request)	Ingress + Istio

설정 관리

	Spring Cloud Config	ConfigMap + Secret
별도 서비스	Config Server 운영 필요	**불필요**
언어	Spring 전용	**언어 무관**
변경 반영	Cloud Bus + RabbitMQ	Reloader or Rolling Restart
감사	커스텀 로깅	**Git 히스토리 (GitOps)**
장애 시	Config Server 다운 = 앱 시작 불가	etcd 기반

상태 확인 & 자가 치유

	Actuator + Eureka	K8s Probes
감지 → 조치	수동 모니터링 + 스크립트	**kubelet 자동 재시작**
감지 시간	~90초 (heartbeat 3회)	**5~10초**
부분 장애	구분 없음	**Liveness vs Readiness**

K8s Probe 구분

```
Liveness Probe 실패          Readiness Probe 실패
        |                            |
        ▼                            ▼
 컨테이너 재시작              Service에서 제거 (재시작 X)
 (프로세스 죽음)              (일시적 장애: DB 연결 끊김 등)
```

핵심: Readiness 실패 = 트래픽만 제거, 재시작 안 함
DB 일시 장애 시 불필요한 재시작 방지

스케일링

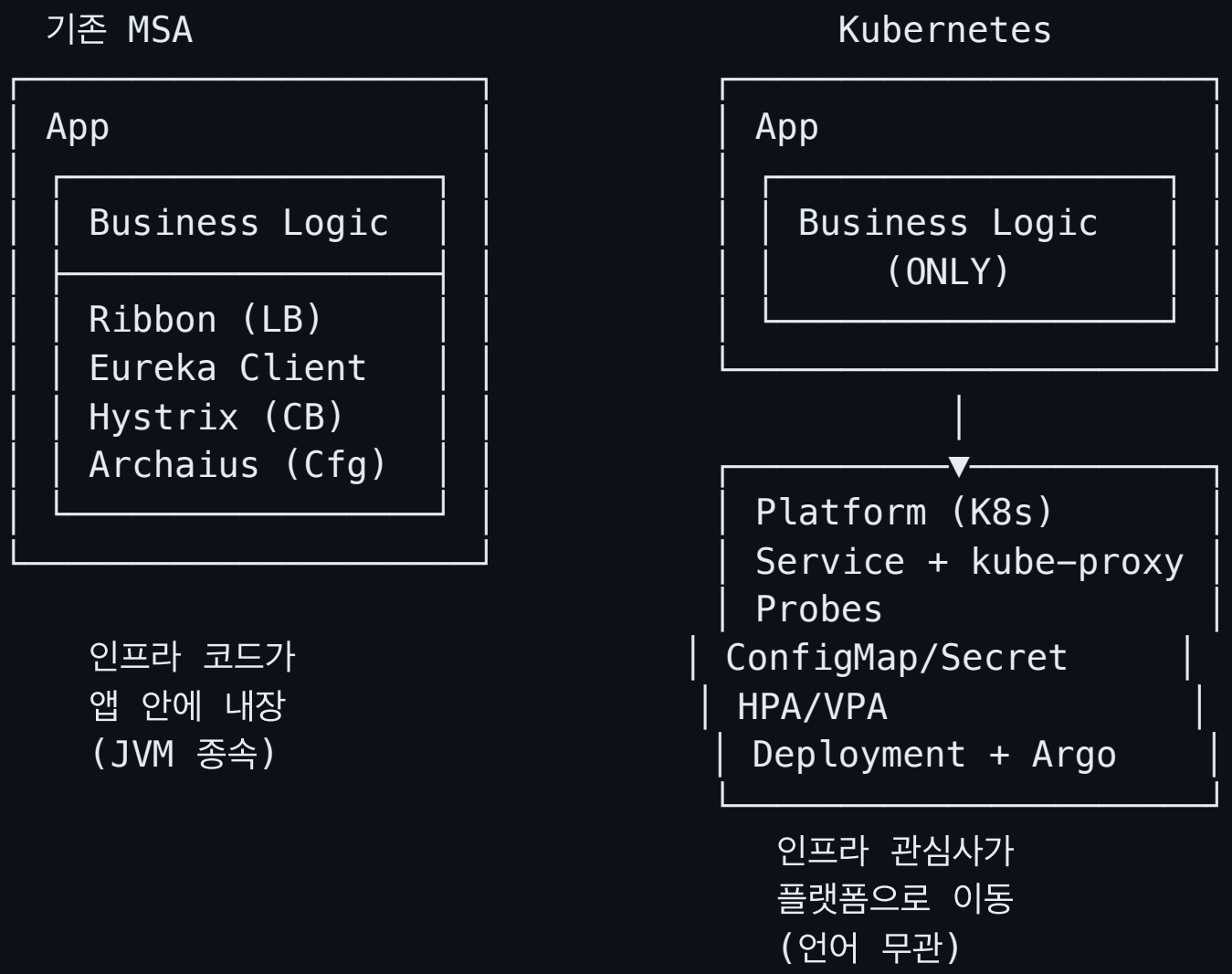
	VM 기반 (ASG)	K8s HPA
단위	VM (분 단위)	Pod (초 단위)
트리거	CPU만	CPU, 메모리, 커스텀 메트릭
Scale-to-zero	불가	KEDA로 가능
세분성	VM 전체	서비스(Pod) 단위

```
HPA
 │ CPU > 70%?
 ▼
┌──────────────┐
│  Replicas    │
│  2 → 3 → 5   │   ← 초 단위 스케일링
└──────────────┘
       │
   ┌───┴───┬───┬───┐
   ▼       ▼   ▼   ▼
 Pod 1   Pod 2 Pod 3 ...
```

배포 전략

	Spinnaker/Jenkins	K8s + Argo
Rolling Update	커스텀 파이프라인	**내장, 선언적**
운영 비용	Spinnaker **8+ 서비스**	Argo = **CRD 1개**
선언적	명령형 파이프라인	**YAML in Git**
Drift 감지	없음	**ArgoCD 지속 동기화**
Rollback	수동	`kubectl rollout undo`

핵심 변화: 인프라 관심사의 이동



요약

Chapter 1 핵심 정리

1. **Monolith → Microservices**: 독립 배포 가능하지만, 운영 복잡도 급증
2. **Container**: VM보다 가볍게 격리 제공 (Namespace + cgroups)
3. **Docker**: Build → Ship → Run 표준화
4. **Control Plane**: API Server, etcd, Scheduler, Controller Manager
5. **Managed K8s**: EKS/GKE가 CP 관리 → 사용자는 Worker만 관리
6. **선언적 모델**: "이 상태를 유지해라" → Reconciliation Loop
7. **서비스 디스커버리**: DNS 캐시/Eureka SPOF → CoreDNS + kube-proxy로 해결

> 기존 MSA는 인프라 관심사를 **앱 라이브러리에 내장** (JVM 종속)
> K8s는 이를 **플랫폼 계층으로 이동** (언어 무관)

Before vs After — 한눈에 보기

관심사	Before (앱 내부)	After (K8s 플랫폼)
Discovery	Eureka Client	CoreDNS
LB	Ribbon	kube-proxy (IPVS)
Config	Config Server	ConfigMap/Secret
Health	Actuator + 수동	Liveness/Readiness
Scaling	ASG (분)	HPA/KEDA (초)
Deploy	Spinnaker (8 svc)	Deployment + Argo

Q&A

Thank you!